More Scheme Lists, Scheme Eval/Apply Solutions

Computer Science Mentors 61A

April 14, 2026 – April 17, 2026

1 Scheme Lists

1. Identify the bug(s) in this program.

```
> (sum-every-other '(1 2 3))
4
> (sum-every-other '())
0
> (sum-every-other '(1 2 3 4))
4
> (sum-every-other '(1 2 3 4 5))
9
```

```
(define (sum-every-other lst)
    (cond ((null? lst) lst)
          (else (+ (cdr lst)
                   (sum-every-other (car lst))))))
```

- The base case should return `0`, not `'()`.
- `(cdr lst)` is a list, so it doesn't make sense to add it to something. Instead, use `(car lst)`, which will give us a number.
- Using `car` (first of the first) is incorrect because the first is a number and sum-every-other takes in a list. Instead, we should use `(cdr (cdr lst))` to skip forward two elements. However, the `cdr` could be `'()`, so we need to add a case to our `cond` to take care of this.

```
(define (sum-every-other lst)
    (cond ((null? lst) 0)
          ((null? (cdr lst)) (car lst))
          (else (+ (car lst)
                   (sum-every-other (cddr lst)) ))))
```

Created by Anaisha Das and Tavisha Patel. Worksheet feedback: forms.gle/q67BZrHHfoKbF6V28

2. Implement `waldo`. `waldo` returns `#t` if the symbol waldo is in a list.

```
scm> (waldo '(1 4 waldo))
#t
scm> (waldo '())
#f
scm> (waldo '(1 4 9))
#f
```

```
(define (waldo lst))
```

```
(define (waldo lst)
    (cond ((null? lst) #f)
          ((eq? (car lst) 'waldo) #t)
          (else (waldo (cdr lst)))
      )
  )
```

Alternate solution:

```
(define (waldo lst)
    (if (null? lst)
        #f
        (if (eq? (car lst) 'waldo)
            #t
            (waldo (cdr lst))
        )
    )
)
```

Similar to how we focus just think about first and rest when working with linked lists, we want to think about solving this question in terms of car and cdr in scheme. At a high level, we can check we can check (car lst) is equal to waldo. If it is, we can return true. Otherwise, we check the rest of the list: (cdr lst). Last, we conclude that waldo is not in the list when we have checked every element but still not found at match.

Observe that the second doctest is essentially the simplest input we can give. From here, we get our first base case– if we are given an empty Scheme list, then return `#f`. Otherwise, notice that regardless of where in the list we find 'waldo', our function should return `#t`. So our function can just stop after we find the first instance of 'waldo in the list, if it exists. Therefore, our second base case checks if our current element (the car of lst) is 'waldo and returns `#t` if this is true. Finally, if neither base case is satisfied, then we must search through the rest of lst, excluding the first element we just checked, for the string 'waldo. So, we make a recursive call using the cdr of lst.

Many problems that involve traversing through a Scheme list will have a similar structure to this solution. Recursive calls on (cdr lst) are also common, similar to recursive calls on link.rest. Note that you can also write a solution using two nested ifs instead of cond, but cond is often a little cleaner to read.

3. **Extra challenge:** Define `waldo` so that it returns the index of the list where the symbol waldo was found (if waldo is not in the list, return `#f`).

```
scm> (waldo '(1 4 waldo))
2
scm> (waldo '())
#f
scm> (waldo '(1 4 9))
#f
```

```
(define (waldo lst)









)
```

```
(define (waldo lst)
    (define (helper lst index)
        (cond ((null? lst) #f)
              ((eq? (car lst) 'waldo) index)
              (else (helper (cdr lst) (+ index 1)))
        )
    )
    (helper lst 0)
)
```

Recall HW2, problem 3, "pingpong" without assignment. To get around the assignment restriction, we introduced a new variable to keep track of state. We passed some state into a helper function to keep track of our iteration. In this problem, we will use the same idea for our implementation.

Inside our recursive helper function, we use the same logic as the original waldo question. Specifically, if the list is empty, we return false. If the first element we see is equal to waldo, we now return the index we have been tracking instead of true. Finally, to recursively search, we call the helper function on the rest of the list and increment the index by 1, returning the result of that call. We call this helper function initially with arguments 'lst' and index 0 to start from the beginning of the list.

A logical alternate solution would be to increment 1 after the recursive call to helper (without incrementing index), instead of adding one to the index, and then recursing. This solution would actually be incorrect. When we increment by one, if the element is not found, `#f` would not be properly returned. At the end of recursion, we would try to add 1 to `#f`, which results in an error.

4. (a) Define `append`, which takes in two lists and returns a new list with all the elements of the first list followed by all the elements of the second. Do not use the built-in `append` function.

```
> (append '(1 2 3) '(4 5 6))
(1 2 3 4 5 6)

(define (append lst1 lst2)









)
```

```
(define (append lst1 lst2)
    (if (null? lst1) lst2
        (cons (car lst1) (append (cdr lst1) lst2))))
```

(b) Define `reverse`. Hint: use `append`.

```
> (reverse '(1 2 3))
(3 2 1)

(define (reverse lst)









)
```

```
(define (reverse lst)
    (if (null? lst) lst
        (append (reverse (cdr lst)) (list (car lst)))))
```

2 Scheme Eval/Apply and Programs as Data

1. What will Scheme output?

```
scm> (define c 4)

c

scm> ((define (x) 1))

Error: str is not callable: x

scm> (x)

1

scm> ((lambda (x y) (+ c)) 1 2)

4

scm> (eval 'c)

2

scm> '(cons 1 nil)

(cons 1 nil)

scm> (eval (list 'if '(even? c) 1 2))

1

scm> (let ((a (+ 3 1)) (b 3)) (+ a b) (/ a b))

1.333333333333333
```

2. Circle or write the number of calls to `scheme_eval` and `scheme_apply` for the code below.

```
(if 1 (+ 2 3) (/ 1 0))
```

scheme_eval	1	3	4	6
scheme_apply	1	2	3	4

6 `scheme_eval`, 1 `scheme_apply`. Evals: (1) on the entire expression, (2) on `1` (**`if`** is not evaluated), (3) on (+ 2 3), (4-6) on +, 2, 3. Apply: (1) with applying + on (+ 2 3).

```
(or #f (and (+ 1 2) 'apple) (- 5 2))
```

scheme_eval	6	8	9	10
scheme_apply	1	2	3	4

8 `scheme_eval`, 1 `scheme_apply`.

```
(define (square x) (* x x))

(+ (square 3) (- 3 2))
```

scheme_eval	2	5	14	24
scheme_apply	1	2	3	4

14 `scheme_eval`, 4 `scheme_apply`.

```
(define (add x y) (+ x y))

(add (- 5 3) (or 0 2))
```

13 `scheme_eval`, 3 `scheme_apply`.

3 Optional

1. Write a function plan-coffee-tour that takes two lists of Berkeley coffee shops and creates an optimized tour according to the following rules: The function creates a tour by alternating shops from each list (similar to interleaving) If a coffee shop appears in both lists, it should only be visited once in the tour at its first occurrence If one list is longer than the other, the remaining unique shops should be added to the end of the tour

```
scm> (plan-coffee-tour '(binge strada philz) '(philz blue-bottle
   binge))
(binge philz strada blue-bottle)

scm> (plan-coffee-tour '(strada mind peets) '(elaichi-co philz))
(strada elaichi-co mind philz peets)

scm> (plan-coffee-tour '(strada qargo) '(strada qargo peets))
(strada qargo peets)

scm> (plan-coffee-tour '() '(delah signal))
(delah signal)

(define (plan-coffee-tour lst1 lst2)
    (cond ((_____________________________) lst2)
        ((_____________________________) lst1)
        (else
        (let ((first (car lst1))
                (rest1 (cdr lst1))
                (rest2 (_____________________________)))
            (if (_____________________________)
                (cons first (plan-coffee-tour rest1 rest2))
                (cons first (cons (_____________________________)
                                  (plan-coffee-tour rest1
                                     (_____________________________)))))))))


    (define (plan-coffee-tour lst1 lst2)
        (cond ((null? lst1) lst2)
            ((null? lst2) lst1)
            (else
                (let ((first (car lst1))
                    (rest1 (cdr lst1))
                    (rest2 (filter (lambda (shop) (not (eq? shop
                       (car lst1)))) lst2)))
                (if (null? rest2)
                    (cons first (plan-coffee-tour rest1 rest2))
                    (cons first (cons (car rest2)
                                      (plan-coffee-tour rest1
                                         (cdr rest2)))))))))
```